

NAME: TANKISO MASOEBE

PROGRAMMING WITH C++2 WK 6

1,SCENARIO ANALYSIS:

Operator Overloading Scenario: Matrix Application

In a matrix application, operator overloading can be used to make mathematical operations easier and more readable in C++. A matrix is a collection of numbers arranged in rows and columns. Normally, adding two matrices would require a function such as `addMatrix()`. With operator overloading, the `+` operator can be redefined to add two matrix objects directly. For example, if Matrix A and Matrix B contain the same number of rows and columns, the expression `Matrix C = A + B` can automatically add corresponding elements from A and B and store the results in C. Similarly, the `*` operator can be overloaded to perform matrix multiplication. This approach makes the program shorter, clearer, and closer to normal mathematical notation. Operator overloading is useful because it allows programmers to give existing C++ operators specific meanings when working with user-defined classes such as matrices.

2.CONCEPT RESEARCH:

Friend Functions

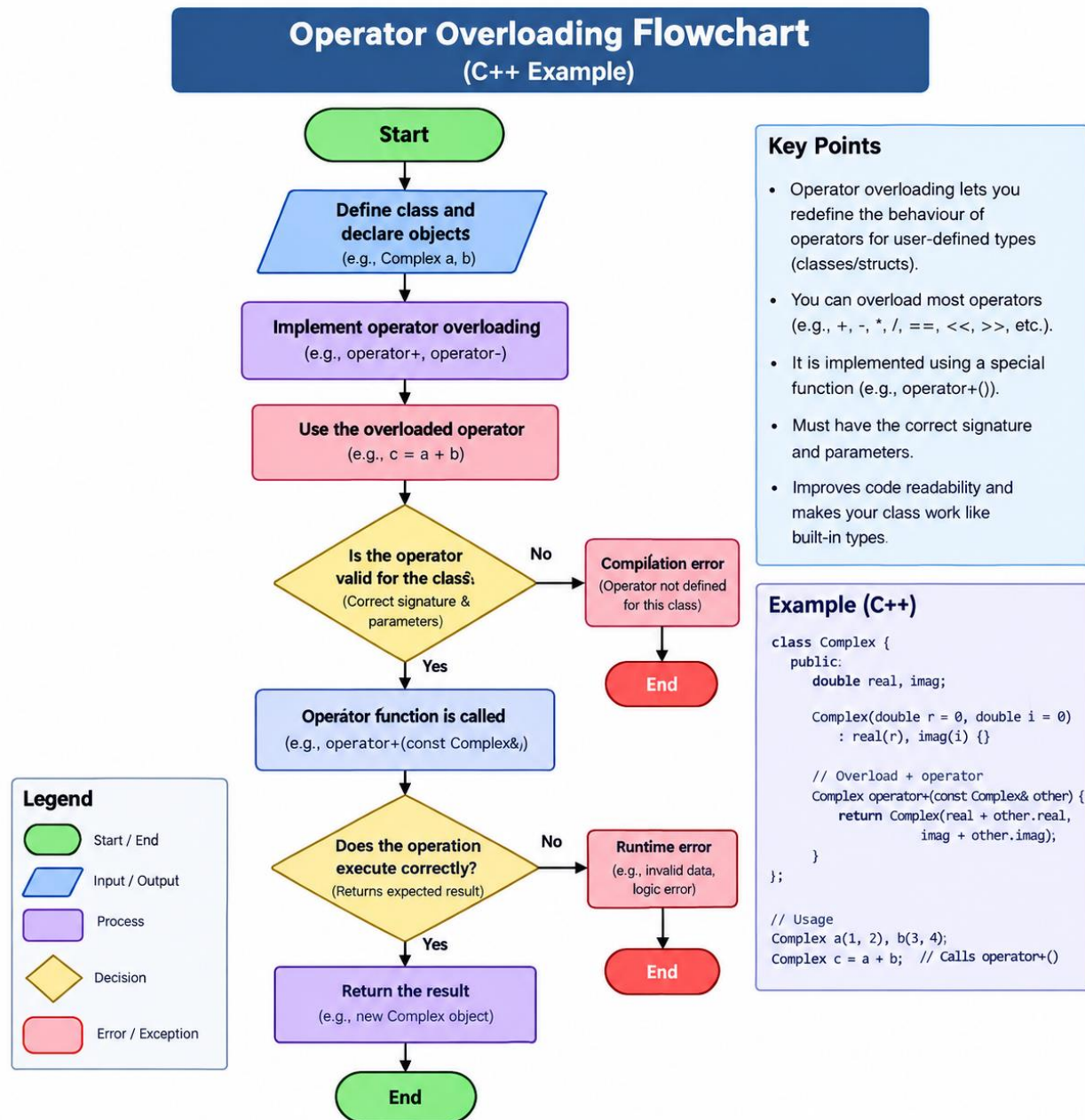
A friend function in C++ is a function that is not a member of a class but is given special permission to access the class's private and protected members. It is declared inside the class using the keyword `friend` and can be defined outside the class. Friend functions are useful when an external function needs to work directly with private data. They are commonly used in operator overloading, where an operator may need access to private members of two different objects. For example, a friend function can compare or add two objects while accessing their private variables. However, friend functions should be used carefully because they reduce data hiding and encapsulation. Overall, they provide controlled access to class data when normal member functions are not suitable.

3.TOOL PRACTICE:

The operator overloading flowchart shows the steps followed when implementing an overloaded operator in C++. First, a class and its objects are created. The programmer then identifies the operator that needs to be overloaded, such as `+` or `<<`. Next, an operator function is defined as either a member function or a friend function. The function performs the required operation using the class data. The program is then compiled and tested. If errors occur, they are corrected before testing again. When the program works correctly, the overloaded operator allows objects to be used with normal operators. For example, overloading `+` can allow two matrix objects to be added using A

+ B. This makes C++ programs easier to understand, shorter, and closer to normal mathematical notation.

4.DIAGRAM:



The operator overloading flowchart shows the steps followed when implementing an overloaded operator in C++. First, a class and its objects are created. The programmer then identifies the operator that needs to be overloaded, such as + or <<. Next, an operator function is defined as either a member function or a friend function. The function performs the required operation using the class data. The program is then

compiled and tested. If errors occur, they are corrected before testing again. When the program works correctly, the overloaded operator allows objects to be used with normal operators. For example, overloading `+` can allow two matrix objects to be added using `A + B`. This makes C++ programs easier to understand, shorter, and closer to normal mathematical notation.